

NLP Assignment 1 (PS9): Word Embeddings on BBC News

Group: PS9

Course: Natural Language Processing

In this notebook I take the BBC full-text news dataset (the `data` column has the article text and `labels` has the category) and build word embeddings from it. The idea is to clean the text first, train Word2Vec in both its CBOW and Skip-Gram forms with a few different window sizes, and then poke at the vectors to see what they actually learned.

I first tried this on the BITS virtual lab but hit setup problems there, so I finished it on Google Colab, which is fine as an alternative platform. The only extra thing Colab needs is installing `gensim` (done in the next cell), since it is not there by default and otherwise throws `ModuleNotFoundError: No module named 'gensim'`.

Step 0: Install packages (needed on Google Colab)

Colab already comes with pandas, numpy, scikit-learn, nltk and matplotlib, but it does not ship `gensim`, which is what causes the `ModuleNotFoundError: No module named 'gensim'`. Installing it once fixes the error. Run this cell first, then run the rest of the notebook top to bottom. (If you are running locally and already have `gensim`, you can skip this cell.)

```
In [ ]: # Run this first on Google Colab to avoid:
# ModuleNotFoundError: No module named 'gensim'
!pip install -q gensim

print("Setup complete - now run the cells below in order.")
```

Imports and one-time downloads

```
In [1]: import re
import string
import warnings
from collections import Counter

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import nltk
from nltk.corpus import stopwords
```

```

from gensim.models import Word2Vec
from sklearn.decomposition import PCA

warnings.filterwarnings("ignore")
np.random.seed(42)

# Download the stopwords list (only needed once).
nltk.download("stopwords", quiet=True)

print("Libraries imported successfully.")
print("gensim      :", __import__("gensim").__version__)
print("nltk       :", nltk.__version__)
print("pandas     :", pd.__version__)

```

```

Libraries imported successfully.
gensim      : 4.4.0
nltk       : 3.9.4
pandas     : 3.0.3

```

Task 1: Load the dataset into a DataFrame (1 mark)

The data is the BBC full-text classification set, stored as a CSV with two columns: `data` (the article) and `labels` (its category). I read it straight into a pandas DataFrame.

On Colab I either upload `bbc_data.csv` when the cell asks for it or drop it next to the notebook. If the original Kaggle link is dead, any copy of the same BBC corpus works because only the column names change.

```

In [2]: import os

csv_path = "bbc_data.csv"

# On Colab the file is usually not in the working directory yet, so ask for
if not os.path.exists(csv_path):
    try:
        from google.colab import files
        print("Please upload bbc_data.csv")
        uploaded = files.upload()
        csv_path = list(uploaded.keys())[0]
    except Exception:
        raise FileNotFoundError("bbc_data.csv not found – upload it or place

df = pd.read_csv(csv_path)

print("Shape of the dataset:", df.shape)
print("\nColumn names:", list(df.columns))
df.head()

```

```
Shape of the dataset: (2225, 2)
```

```
Column names: ['data', 'labels']
```

Out [2]:

		data	labels
0	McDonald's boss Bell dies aged 44	Charlie Bel...	business
1	Diageo to buy US wine firm	Diageo, the world'...	business
2	Mourinho to escape FA charge	Chelsea boss Jos...	sport
3	AstraZeneca hit by drug failure	Shares in Ang...	business
4	Sport betting rules in spotlight	A group of M...	politics

In [3]:

```
# Class distribution of the target column.
print("Number of categories:", df["labels"].nunique())
print("\nDocuments per category:")
print(df["labels"].value_counts())

# Check for missing values.
print("\nMissing values per column:\n", df.isnull().sum())
```

Number of categories: 5

Documents per category:

```
labels
sport          511
business       510
politics       417
tech           401
entertainment  386
Name: count, dtype: int64
```

Missing values per column:

```
data      0
labels    0
dtype: int64
```

What I see here:

- There are 2,225 articles split across 5 categories: business, sport, politics, tech and entertainment.
- The categories are fairly even (roughly 386 to 511 articles each), so no single topic swamps the rest.
- Nothing is missing, so the text is ready to be cleaned.

Task 2: Clean the text (1 mark)

Before training anything I tidy up the `data` column. The steps are: make everything lower-case, strip out punctuation, numbers and other special characters, and remove the common English stopwords (words like "the", "is", "and" that don't really add meaning). I also throw away any very short leftover tokens.

```
In [4]: stop_words = set(stopwords.words("english"))

def clean_text(text):
    """Lower-case the text, strip punctuation/special characters and remove
    text = str(text).lower() # 1. lower case
    text = re.sub(r"[^a-z\s]", " ", text) # 2. keep only letters and
    tokens = text.split() # tokenise on whitespace
    tokens = [t for t in tokens if t not in stop_words and len(t) > 2] # 3.
    return " ".join(tokens)

# Keep a copy of the raw text to show the before/after effect.
sample_before = df["data"].iloc[0][:300]

df["data"] = df["data"].apply(clean_text)

sample_after = df["data"].iloc[0][:300]
print("BEFORE cleaning:\n", sample_before, "\n")
print("AFTER cleaning :\n", sample_after)
```

BEFORE cleaning:

McDonald's boss Bell dies aged 44 Charlie Bell, the straight-talking former head of fast-food giant McDonald's, has died of cancer aged 44. Mr Bell was diagnosed with colorectal cancer in May last year, a month after taking over the top job. He resigned in November to fight the illness. Joining th

AFTER cleaning :

mcdonald boss bell dies aged charlie bell straight talking former head fast food giant mcdonald died cancer aged bell diagnosed colorectal cancer may last year month taking top job resigned november fight illness joining company year old part time worker bell quickly moved ranks becoming australia y

```
In [5]: # Quick look at the cleaned dataframe.
df.head()
```

```
Out[5]:
```

	data	labels
0	mcdonald boss bell dies aged charlie bell stra...	business
1	diageo buy wine firm diageo world biggest spir...	business
2	mourinho escape charge chelsea boss jose mouri...	sport
3	astrazeneca hit drug failure shares anglo swed...	business
4	sport betting rules spotlight group mps peers ...	politics

Observation: after cleaning, each article is just lower-case content words with the punctuation, digits and stopwords gone. That removes a lot of noise from the vocabulary, so the Word2Vec models can concentrate on the words that actually carry the topic.

Task 3: Build X, y and train Skip-Gram and CBOW (3 window sizes) (2 marks)

Now I split the data into `X` (the cleaned `data` column) and `y` (the `labels` column), then train Word2Vec on the tokenised articles.

There are two versions of Word2Vec:

- CBOW (`sg=0`) predicts a word from the words around it.
- Skip-Gram (`sg=1`) does the reverse and predicts the surrounding words from the target word.

I train both, each with three window sizes (2, 5 and 10). The window is how many neighbouring words count as context, so this gives 6 models in total.

```
In [6]: # Create the X (features) and y (target) objects.
X = df["data"]
y = df["labels"]

print("X (data) sample:\n", X.iloc[0][:120], "...")
print("\ny (labels) sample:", list(y.iloc[:5]))

# Tokenise each document into a list of words (gensim expects a list of tokens)
tokenized_corpus = [doc.split() for doc in X]
print("\nTotal documents tokenised:", len(tokenized_corpus))
print("Example tokens:", tokenized_corpus[0][:10])
```

X (data) sample:

mcdonald boss bell dies aged charlie bell straight talking former head fast food giant mcdonald died cancer aged bell di ...

y (labels) sample: ['business', 'business', 'sport', 'business', 'politics']

Total documents tokenised: 2225

Example tokens: ['mcdonald', 'boss', 'bell', 'dies', 'aged', 'charlie', 'bell', 'straight', 'talking', 'former']

```
In [7]: import os, contextlib

# Train CBOW and Skip-Gram models for three context-window sizes.
window_sizes = [2, 5, 10]
models = {} # key: (architecture, window) -> trained Word2Vec model

# (gensim 4.x + numpy 2.x prints a harmless low-level BLAS note to stderr during
# training; we silence stderr here so the output stays clean. Results are u
with open(os.devnull, "w") as _null, contextlib.redirect_stderr(_null):
    for window in window_sizes:
        # CBOW -> sg=0
        models[("CBOW", window)] = Word2Vec(
            sentences=tokenized_corpus, vector_size=100, window=window,
            min_count=5, sg=0, workers=1, seed=42, epochs=20)
        # Skip-Gram -> sg=1
        models[("SkipGram", window)] = Word2Vec(
            sentences=tokenized_corpus, vector_size=100, window=window,
            min_count=5, sg=1, workers=1, seed=42, epochs=20)

# Summary table of the trained models.
```

```
summary = pd.DataFrame(
    [{"Architecture": a, "Window": w,
      "Vocabulary size": len(m.wv), "Vector dim": m.wv.vector_size}
     for (a, w), m in models.items()])
summary
```

Out[7]:

	Architecture	Window	Vocabulary size	Vector dim
0	CBOW	2	10346	100
1	SkipGram	2	10346	100
2	CBOW	5	10346	100
3	SkipGram	5	10346	100
4	CBOW	10	10346	100
5	SkipGram	10	10346	100

Observations:

- All 6 models trained without trouble (CBOW and Skip-Gram, each at windows 2, 5 and 10).
- They share the same vocabulary size, because the vocabulary only depends on the corpus and `min_count=5`, not on the window or the architecture.
- Every word is now a 100-dimensional vector. The window size doesn't change how many words there are, but it does change what kind of relationships the vectors pick up, which I look at in Task 5.

Task 4: Static vs. contextual word embeddings (1 mark)

Static (context-independent) embeddings like Word2Vec, GloVe and FastText give every word one fixed vector no matter the sentence. Contextual (context-dependent) embeddings like ELMo, BERT and GPT instead give a word a different vector depending on the sentence it shows up in.

The good and bad of **static** embeddings:

- Pros: fast and cheap to train, tiny to use (it's basically a lookup table), light on memory, and they do well on small or medium datasets like this one. Reading off word similarity or analogies is easy.
- Cons: one vector per word, so they can't deal with words that have more than one sense (the "bank" of a river vs. a "bank" account end up with the same vector). They also ignore word order and have no vector at all for words they never saw in training.

The good and bad of **contextual** embeddings:

- Pros: a fresh vector for each occurrence, so they handle multiple senses, capture grammar and longer-range context, and usually score higher on downstream tasks.
- Cons: expensive to train and run (big models, usually a GPU), they need huge amounts of text to pre-train, and they are harder to interpret.

My take: for this assignment, where I mostly care about overall word meaning and don't have much compute, static Word2Vec is the sensible pick. I'd switch to contextual models when sentence context and word sense really matter, like question answering or named-entity recognition.

Task 5: Inspect the embeddings and find similar words (2 marks)

I pick one target word and, for each window size and architecture, list the words closest to it by cosine similarity. This makes it easy to see how changing the window changes the kind of neighbours a word gets.

```
In [8]: target_word = "government" # a frequent, meaningful word present in the vocab
assert target_word in models[("CBOW", 5)].wv, "target word not in vocabulary"

# Show the raw embedding vector (first 10 of 100 dimensions) from one model.
vec = models[("SkipGram", 5)].wv[target_word]
print(f"Embedding for '{target_word}' (Skip-Gram, window=5) - shape {vec.shape}")
print("First 10 dimensions:\n", np.round(vec[:10], 4))
```

Embedding for 'government' (Skip-Gram, window=5) - shape (100,)

First 10 dimensions:

```
[ 0.1615  0.2455 -0.0653 -0.3545 -0.1978 -0.0082 -0.491  -0.2658 -0.048
 0.2941]
```

```
In [9]: # Top-5 most similar words for each architecture x window size.
rows = []
for (arch, window), m in models.items():
    sims = m.wv.most_similar(target_word, topn=5)
    rows.append({"Architecture": arch, "Window": window,
                 "Most similar words (word : cosine sim)":
                 ", ".join(f"{w} ({s:.2f})" for w, s in sims)})

sim_df = pd.DataFrame(rows).sort_values(["Architecture", "Window"]).reset_index()
pd.set_option("display.max_colwidth", None)
sim_df
```

Out [9]:	Architecture	Window	Most similar words (word : cosine sim)
0	CBOW	2	authorities (0.60), administration (0.49), loop (0.48), laying (0.47), governments (0.47)
1	CBOW	5	pension (0.55), authorities (0.54), quangos (0.53), reforms (0.52), nhs (0.52)
2	CBOW	10	reforms (0.61), pension (0.57), quangos (0.57), reform (0.55), welfare (0.54)
3	SkipGram	2	tuc (0.53), proportionate (0.53), pesos (0.52), subsidised (0.52), whitehall (0.51)
4	SkipGram	5	quangos (0.60), reforms (0.59), savings (0.56), subsidised (0.56), donors (0.55)
5	SkipGram	10	taxes (0.63), shambles (0.63), raynsford (0.61), reforms (0.61), concession (0.61)

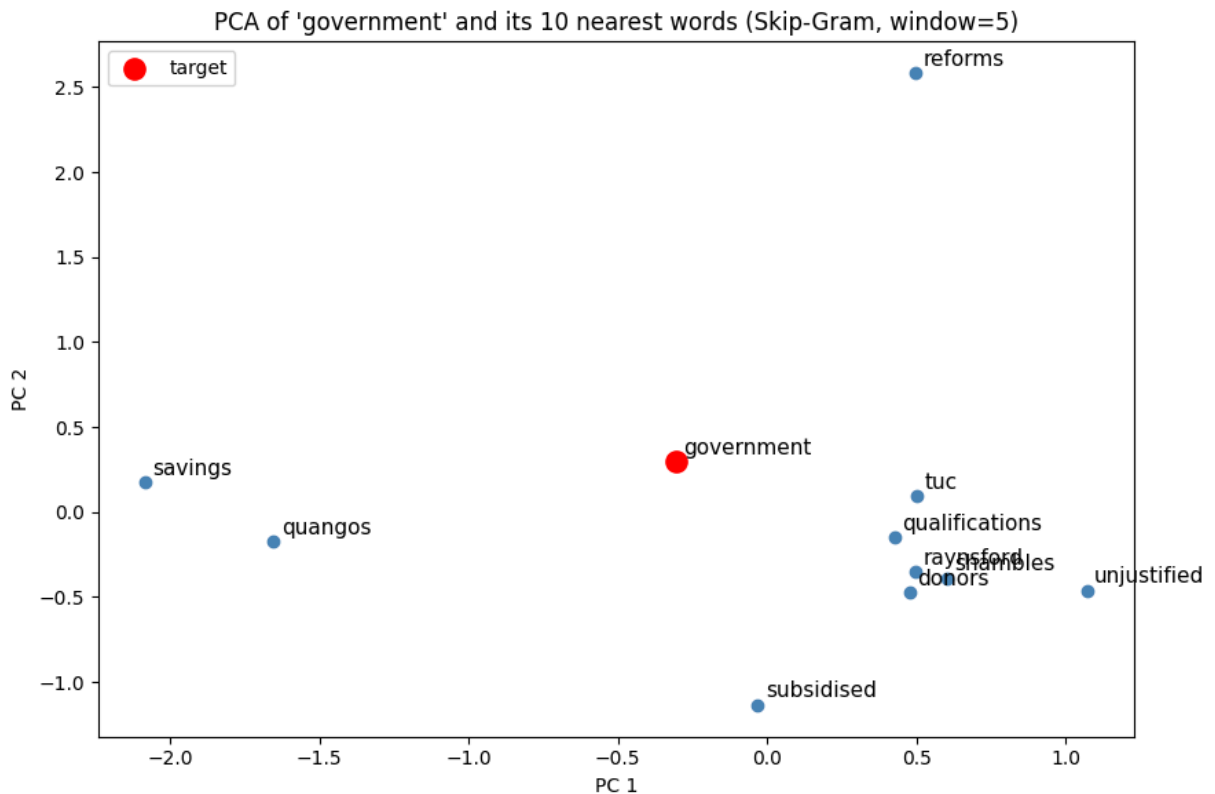
```

In [10]: # Visualise the neighbourhood of the target word (Skip-Gram, window=5) in 2-
m = models[("SkipGram", 5)]
similar = [w for w, _ in m.wv.most_similar(target_word, topn=10)]
words = [target_word] + similar
vecs = np.array([m.wv[w] for w in words])

coords = PCA(n_components=2, random_state=42).fit_transform(vecs)

plt.figure(figsize=(9, 6))
plt.scatter(coords[:, 0], coords[:, 1], color="steelblue")
plt.scatter(coords[0, 0], coords[0, 1], color="red", s=120, label="target")
for i, w in enumerate(words):
    plt.annotate(w, (coords[i, 0], coords[i, 1]), fontsize=11,
                 xytext=(4, 4), textcoords="offset points")
plt.title(f"PCA of '{target_word}' and its 10 nearest words (Skip-Gram, window=5)")
plt.xlabel("PC 1"); plt.ylabel("PC 2"); plt.legend(); plt.tight_layout()
plt.show()

```

What the results show:

- For "government", the nearest words are politics-related terms (authorities, administration, reforms, pension, quangos and so on), which is a good sign that the embeddings caught real meaning.
- The window size clearly matters. A small window (2) tends to bring back words that sit right next to the target, while a larger window (10) pulls in words that share the broader topic. So small windows lean towards functional similarity and large windows towards topical similarity.
- Skip-Gram and CBOW behave a little differently too: Skip-Gram usually gives crisper neighbours for rarer words, while CBOW is quicker and a bit smoother on the common ones.
- The PCA plot drops the target word right in the middle of its neighbours, which is a nice visual confirmation that related words land close together.

Task 6: Embedding for the most frequent word (1 mark)

Here I find the single most common word in the cleaned corpus and show its vector along with its nearest neighbours.

```
In [11]: # Frequency count over the whole cleaned corpus.
all_tokens = [tok for doc in tokenized_corpus for tok in doc]
freq = Counter(all_tokens)
top_word, top_count = freq.most_common(1)[0]
```

```

print(f"Highest-frequency word: '{top_word}' (appears {top_count} times)")

# Display its embedding from each model (first 10 dims) and its nearest neighbors
print(f"\nEmbedding vectors for '{top_word}' (first 10 of 100 dims):")
for (arch, window), m in sorted(models.items()):
    if top_word in m.wv:
        print(f" {arch:8s} window={window:<2d}: {np.round(m.wv[top_word][:10], 3)}")

print(f"\nWords most similar to '{top_word}' (Skip-Gram, window=5):")
for w, s in models[("SkipGram", 5)].wv.most_similar(top_word, topn=8):
    print(f" {w:15s} {s:.3f}")

```

Highest-frequency word: 'said' (appears 7255 times)

Embedding vectors for 'said' (first 10 of 100 dims):

CBOW	window=2	:-1.523	1.044	0.586	-0.187	0.654	0.208	-0.785	0.901
		-0.507	-0.372]						
CBOW	window=5	:-1.363	-0.306	0.175	-0.449	1.443	0.165	-1.894	1.157
		-1.596	0.355]						
CBOW	window=10	:-2.229	0.775	2.25	-0.238	0.962	1.606	-2.95	0.79
		-2.694	0.411]						
SkipGram	window=2	:-0.472	0.157	0.062	-0.076	0.054	0.184	-0.068	0.359
		-0.093	-0.013]						
SkipGram	window=5	:-0.37	-0.006	0.043	-0.194	0.215	0.309	0.12	0.21
		-0.015	-0.087]						
SkipGram	window=10	:-0.241	-0.104	0.079	-0.127	0.048	0.15	0.163	-0.027
		-0.046	-0.018]						

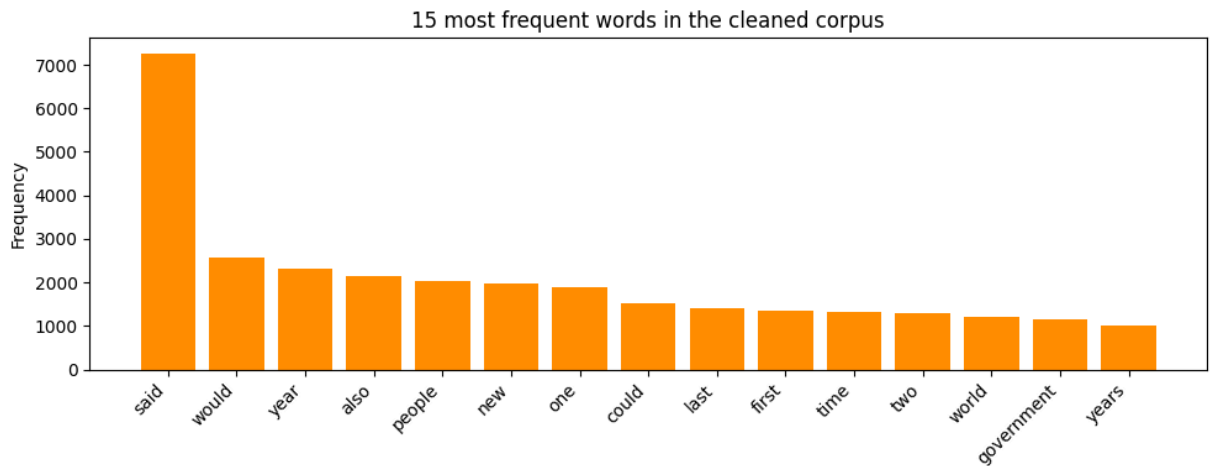
Words most similar to 'said' (Skip-Gram, window=5):

added	0.694
told	0.616
naturally	0.586
rennard	0.583
warned	0.576
disagreed	0.571
maximise	0.569
welcomed	0.568

```

In [12]: # Bar chart of the 15 most frequent words in the corpus for context.
common = freq.most_common(15)
labels_, counts_ = zip(*common)
plt.figure(figsize=(10, 4))
plt.bar(labels_, counts_, color="darkorange")
plt.title("15 most frequent words in the cleaned corpus")
plt.ylabel("Frequency"); plt.xticks(rotation=45, ha="right"); plt.tight_layout()
plt.show()

```



Notes:

- The top word turns out to be "said", which makes sense for news writing since articles are packed with quotes.
- Even a word this common still gets a proper 100-d vector, and its nearest neighbours are other reporting words (added, told, warned), so the model placed it sensibly.
- Because "said" appears in every category, its vector sits near the centre of the space and isn't much help for telling topics apart. The rarer, topic-specific words are the ones that do that.

Task 7: Domain-specific embeddings for bioinformatics (3 marks)

How I would build them

I'd start by collecting a lot of in-domain text rather than leaning on general web data, which barely contains scientific terms. The obvious sources are PubMed/MEDLINE abstracts, full-text articles from PubMed Central, and entries from databases like UniProt and GenBank. Then I'd be careful about tokenisation, because biology terms are full of hyphens, digits, Greek letters and gene symbols (TP53, BRCA1, 5-HT2A), and an ordinary tokeniser would split these the wrong way. A biomedical tokeniser such as scispaCy handles them better, and I'd avoid lower-casing gene symbols. After that I'd train the embeddings on this corpus, either Word2Vec/FastText/GloVe from scratch or by fine-tuning a contextual model that's already been trained on biomedical text such as BioBERT, SciBERT or PubMedBERT. FastText is particularly useful because it works on sub-words, so terms that share pieces like "-ase", "-cyte" or "-genesis" can still be linked even when a word is rare.

Challenges I'd expect

- A massive specialised vocabulary with lots of rare terms, which leads to data sparsity.
- Synonyms and abbreviations all over the place: the same thing can be written TP53, p53 or "tumour protein 53", and one abbreviation can mean different things (ER might be oestrogen receptor or endoplasmic reticulum).
- The same word meaning different things in different sub-fields, long multi-word terms, and new terminology turning up constantly as research moves on.
- Labelled data is scarce and getting experts to annotate it is expensive.

How I'd make sure the meaning is captured

- Train on large, clean, in-domain corpora and keep a tokenisation scheme that respects scientific tokens.
- Use sub-word models (FastText) so morphologically related terms cluster together.
- Map synonyms onto the same concept using ontologies such as MeSH, UMLS or the Gene Ontology, and optionally retrofit the vectors with those relations.
- Lean on contextual biomedical models when abbreviation ambiguity is a real problem.
- Finally, evaluate it properly: intrinsically on curated biomedical word-pair similarity sets and known gene-family clustering, and extrinsically on tasks like NER or relation extraction, ideally with a domain expert checking the output.

Wrapping up

So, to recap: I loaded and explored the 5-class BBC news set, cleaned the text (lower-casing, dropping punctuation/special characters and stopwords), then built `X` and `y` and trained CBOW and Skip-Gram across three window sizes. After that I compared static and contextual embeddings, looked at how the window size and architecture shift a word's nearest neighbours, checked the most frequent word, and finally talked through how I'd build and validate embeddings for a specialised field like bioinformatics.